

Problem Session 08: Lock Mechanisms

Peterson's Algorithm · LL/SC · Lamport's Bakery

CPE 333 Operating Systems, 1/2026 — Group OS InW

Group: OS InW

67070501005

Kiatisak Markmeeshap

Peterson's Algorithm

67070501040

Worawut Sereethai

Lamport's Bakery Algorithm

67070501018

Tithinan Sobking

Load-Linked / Store-Conditional

67070501058

Chanya Poolketkij

Comparison and Conclusion

How We Evaluate Each Lock

- Metrics from Lecture 8:
 - Mutual exclusion: counter must equal total acquisitions (lost = total – counter)
 - Absence of deadlock: every run must finish
 - Fairness: fewest ÷ most acquisitions per thread (1.0 = equal)
 - Performance: acquisitions per second
- 1 s per run, median of 3 runs, 1/2/4/8 threads, on all 8 CPUs and on one CPU (taskset -c 0)
- WSL2 Ubuntu, 8 CPUs, gcc 15.2.0 -O2
- Reference locks from the lecture: TAS spin lock, pthread_mutex

```
while (!stop) {  
    lock_acquire(id);  
    counter = counter + 1;           /* critical section */  
    lock_release(id);  
    n++;  
}
```

Peterson's Algorithm

67070501005

Kiatisak Markmeeshap

Peterson's Algorithm: Idea

- Pure software lock for two threads: only loads and stores
- $\text{flag}[i] = 1$ — “thread i wants to enter”
- $\text{turn} = \text{other}$ — “you go first”
- Wait while the other wants in and it is the other's turn
- The thread that wrote turn last waits

```
static volatile int flag[2];    /* flag[i] = 1: thread i wants in */
static volatile int turn;       /* whose turn it is to wait */

void lock_acquire(int self)
{
    int other = 1 - self;
    flag[self] = 1;             /* I want to enter ... */
    turn = other;               /* ... but you go first */
    FENCE();                   /* stores done before the loads */
    while (flag[other] == 1 && turn == other)
        ;                      /* spin: you want in, your turn */
}

void lock_release(int self)
{
    FENCE();
    flag[self] = 0;
}
```

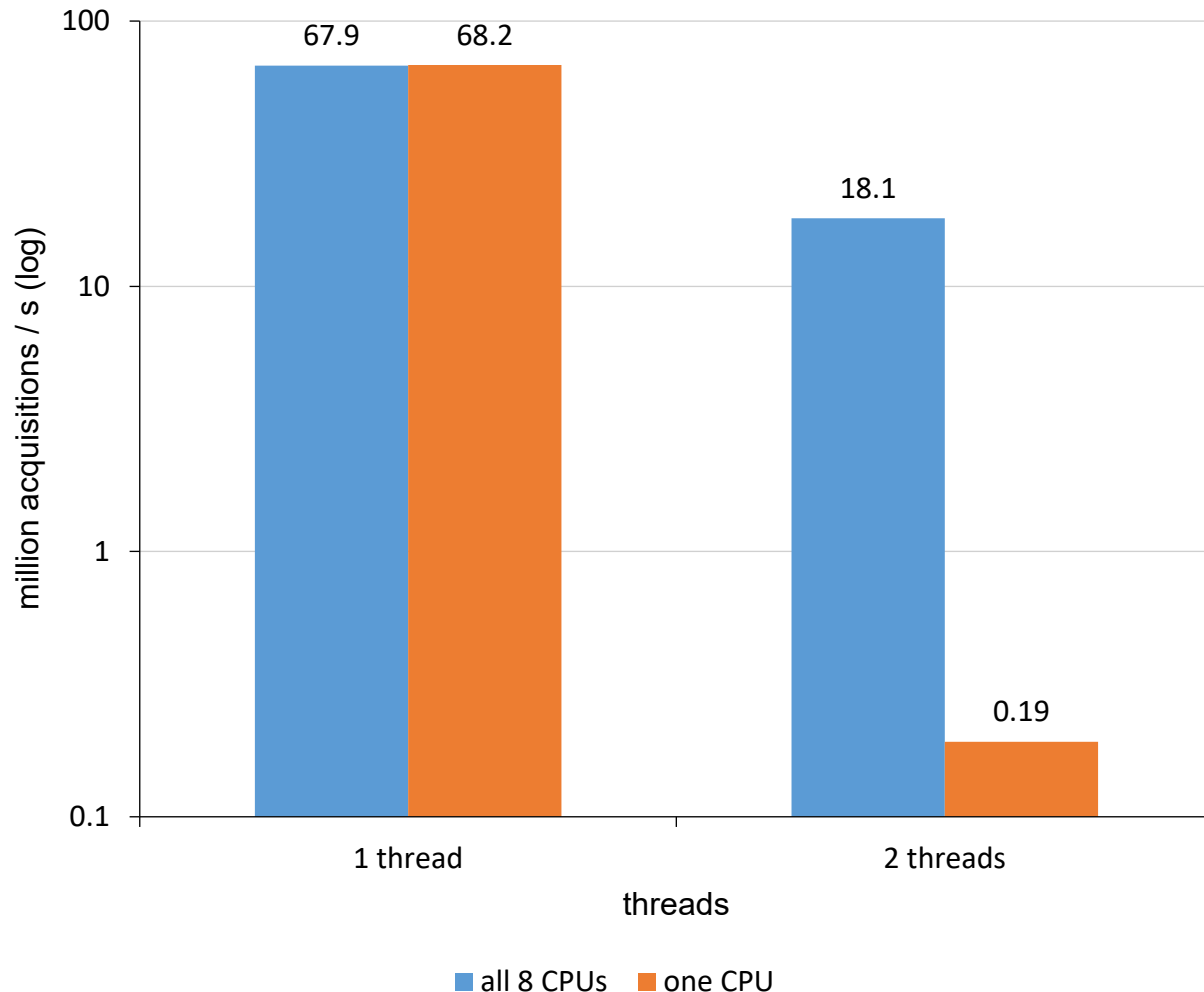
Peterson: Mutual Exclusion and Deadlock

- Mutual exclusion: if both were inside, each saw `turn == itself` — impossible, `turn` holds one value
- No deadlock: `turn` is 0 or 1, so one waiting thread always passes
- But x86 lets a load pass an earlier store (store buffer): both read `flag[other] == 0` and both enter

Build (2 threads)	CPUs	Runs with lost updates	Lost updates per run
no fence	all 8	5 / 5	163,764 – 916,884
no fence	one	0 / 3	0
with FENCE()	all 8	0 / 5	0

- FENCE() compiles to `lock orq $0x0,(%rsp)` — a full barrier: the stores drain before the loads

Peterson: Performance



- 1 thread: 68 M acq/s — no contention
- 2 threads, all CPUs: 18 M acq/s — flag and turn move between cores
- 2 threads, one CPU: 192 k acq/s — the waiter spins until the scheduler switches
- Busy-waiting costs a whole time slice when the other thread is not running

Peterson: Fairness and Limitations

- All CPUs, 2 threads: fairness 0.99 — turn gives way to the other thread, so a waiter enters after at most one entry by the other (bounded waiting)
- One CPU, 2 threads: fairness 0.001 (run 1: 126 vs 192,299) — with spinning, the scheduler decides
- Limitations:
 - Two threads only (N threads need a tournament of Peterson locks or the filter lock)
 - Needs a memory fence on modern CPUs — the textbook version is broken on x86
 - Spins instead of sleeping

Load-Linked / Store-Conditional

67070501018

Tithinan Sobking

LL/SC: Idea

- LoadLinked(p): load *p and watch the address
- StoreConditional(p, v): store only if nobody stored to p since the LoadLinked
- Lock: spin on LL until 0, then SC 1; if SC fails, start over
- MIPS, Alpha, PowerPC, ARM (AArch64: LDXR/STXR)
- x86 has no LL/SC — it offers CAS (lock cmpxchg)

```
static inline int LoadLinked(volatile int *p)
{
    int v;
    __asm__ volatile("ldaxr %w0, [%1]"
                     : "=r"(v) : "r"(p) : "memory");
    return v;
}

static inline int StoreConditional(volatile int *p, int v)
{
    int fail;
    __asm__ volatile("stxr %w0, %w2, [%1]"
                     : "=r"(fail) : "r"(p), "r"(v) : "memory");
    return !fail;
}

void lock_acquire(int id)
{
    (void)id;
    while (1) {
        while (LoadLinked(&flag))
            ; /* spin until it is zero */
        if (StoreConditional(&flag, 1))
            return; /* no store in between: ours */
        /* else another thread won: try again */
    }
}
```

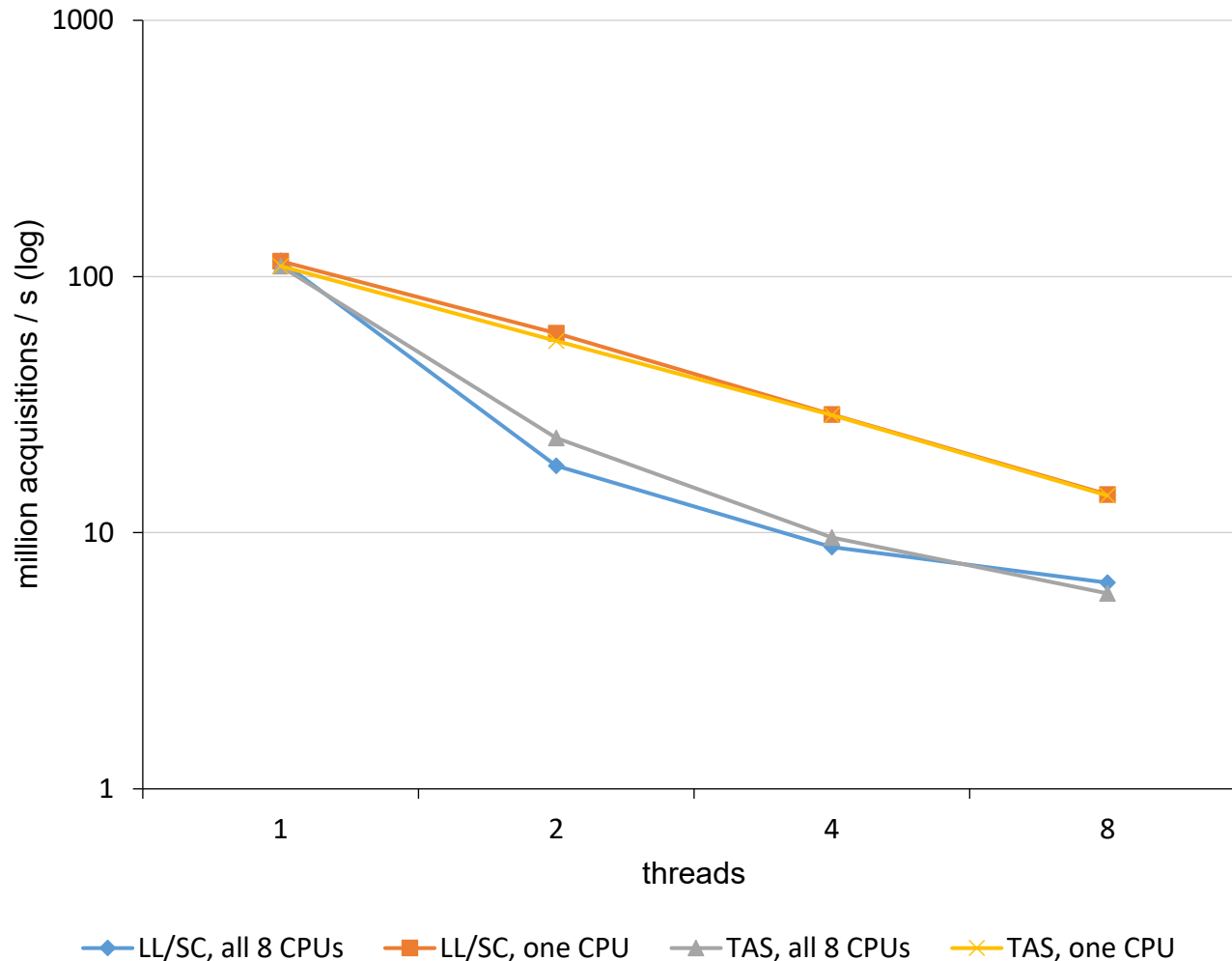
LL/SC: Mutual Exclusion and Deadlock

- Mutual exclusion: two threads may both LL a 0, but the first SC writes the flag, so the second SC fails
- No deadlock: an SC fails only if another store got in (that thread now holds the lock) or spuriously (e.g. an interrupt) — the loop retries
- AArch64 build under qemu-aarch64 10.2.1, 4 threads: lost = 0 in 3 / 3 runs (5,520,491+ acquisitions each)

```
0000000000400b8c <lock_acquire>:  
adrp x1, 4b1000  
add x1, x1, #0xb40  
mov w2, #0x1  
nop  
nop  
ldaxr w0, [x1]  
cbnz w0, 400ba0 <lock_acquire+0x14>  
stxr w0, w2, [x1]  
cbnz w0, 400ba0 <lock_acquire+0x14>  
ret
```

ldaxr = Load-Linked, stxr = Store-Conditional (writes 0 to w0 on success)

LL/SC: Performance



- Timed on x86 with CAS standing in for LL/SC (qemu speed is meaningless)
- All CPUs: 115 M acq/s with 1 thread → 6.4 M with 8 — every try pulls the flag's cache line
- One CPU, 8 threads: 14 M acq/s
- Waiters only read (LL) until the flag looks free, then write once (SC)

LL/SC: Fairness and Limitations

- No queue: the first SC to land wins — fairness at 8 threads 0.45 (all CPUs), 0.00 (one CPU; run 1: the unluckiest thread got 1 of 14,215,110)
- A thread preempted while holding the lock makes every other thread spin out its time slice
- Limitations:
 - Starvation is possible; spins instead of sleeping
 - Hardware-specific: not on x86; SC may fail spuriously
 - Little may run between LL and SC, or the reservation is lost
- Advantage over CAS: SC fails on any store, even one that writes back the same value (no ABA problem)

Lamport's Bakery Algorithm

67070501040

Worawut Sereethai

Bakery Algorithm: Idea

- Take a number: 1 + the largest number in use
- Smallest number enters first; equal numbers → lower thread id first
- choosing[j]: wait until j has finished picking
- N threads, only loads and stores (Lamport, 1974)

```
void lock_acquire(int i)
{
    choosing[i] = 1;
    FENCE();
    unsigned max = 0;
    for (int j = 0; j < n; j++)
        if (number[j] > max) max = number[j];
    number[i] = max + 1;      /* take the next number */
    FENCE();
    choosing[i] = 0;
    FENCE();

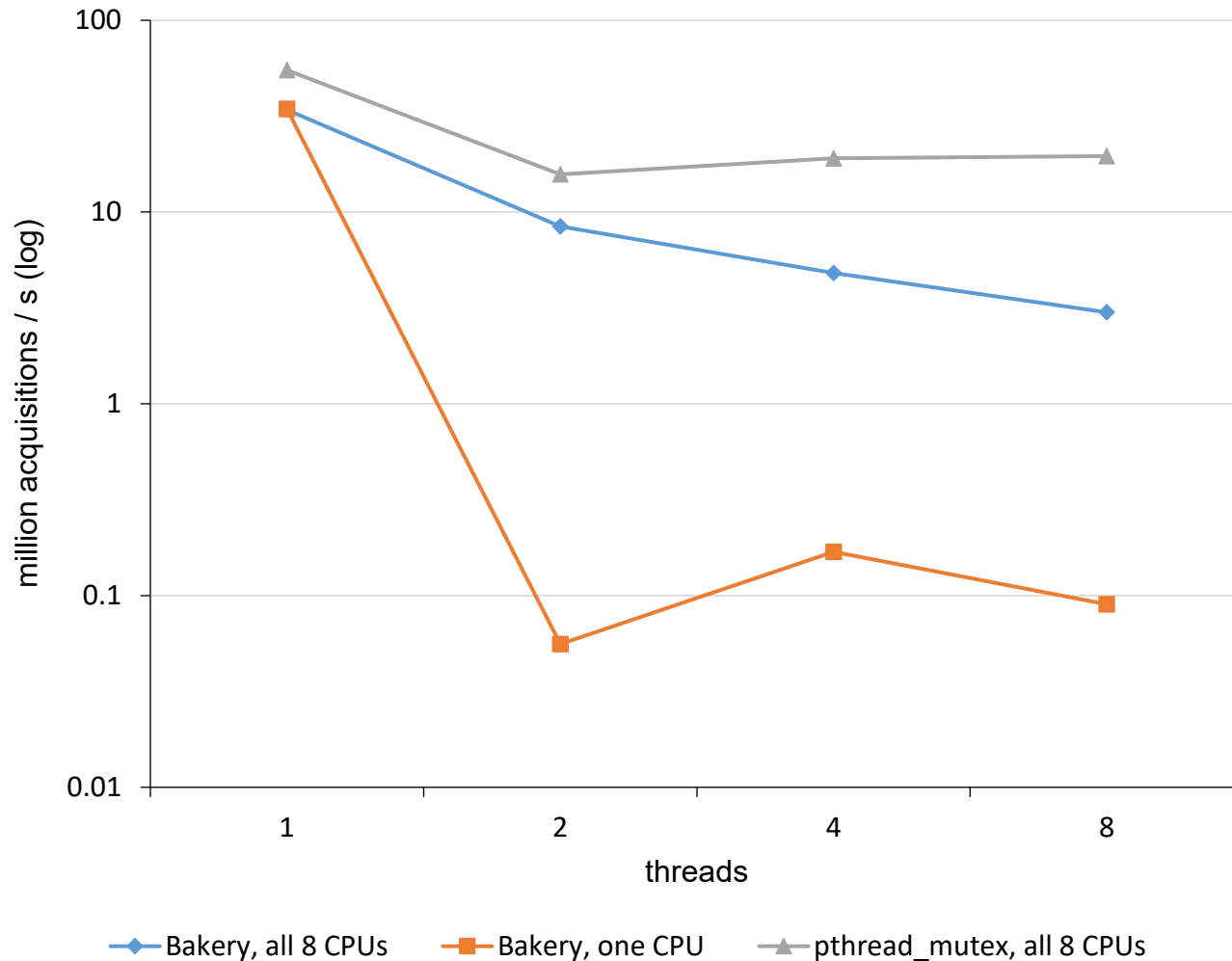
    for (int j = 0; j < n; j++) {
        while (choosing[j])
            ;                /* j is still picking */
        while (number[j] != 0 &&
              (number[j] < number[i] ||
               (number[j] == number[i] && j < i)))
            ;                /* j is ahead of me */
    }
}

void lock_release(int i)
{
    FENCE();
    number[i] = 0;
}
```

Bakery: Mutual Exclusion and Deadlock

- Mutual exclusion: (number, id) pairs are unique and ordered; a thread enters only after every thread with a smaller pair has left
- choosing[] closes the race where j has read the maximum but not yet written its number
- No deadlock: the waiting thread with the smallest pair waits for nobody, so it enters
- Like Peterson, it needs memory fences on x86 (FENCE() in the code)
- Measured: all 24 runs (1–8 threads, all CPUs and one CPU) finished with lost = 0

Bakery: Performance



- lock() scans all N threads twice: $O(N)$ per acquisition
- All CPUs: 8.4 M $\rightarrow$ 4.8 M $\rightarrow$ 3.0 M acq/s at 2/4/8 threads
- One CPU, 8 threads: 90 k acq/s — the next in line must be scheduled first

Bakery: Fairness and Limitations

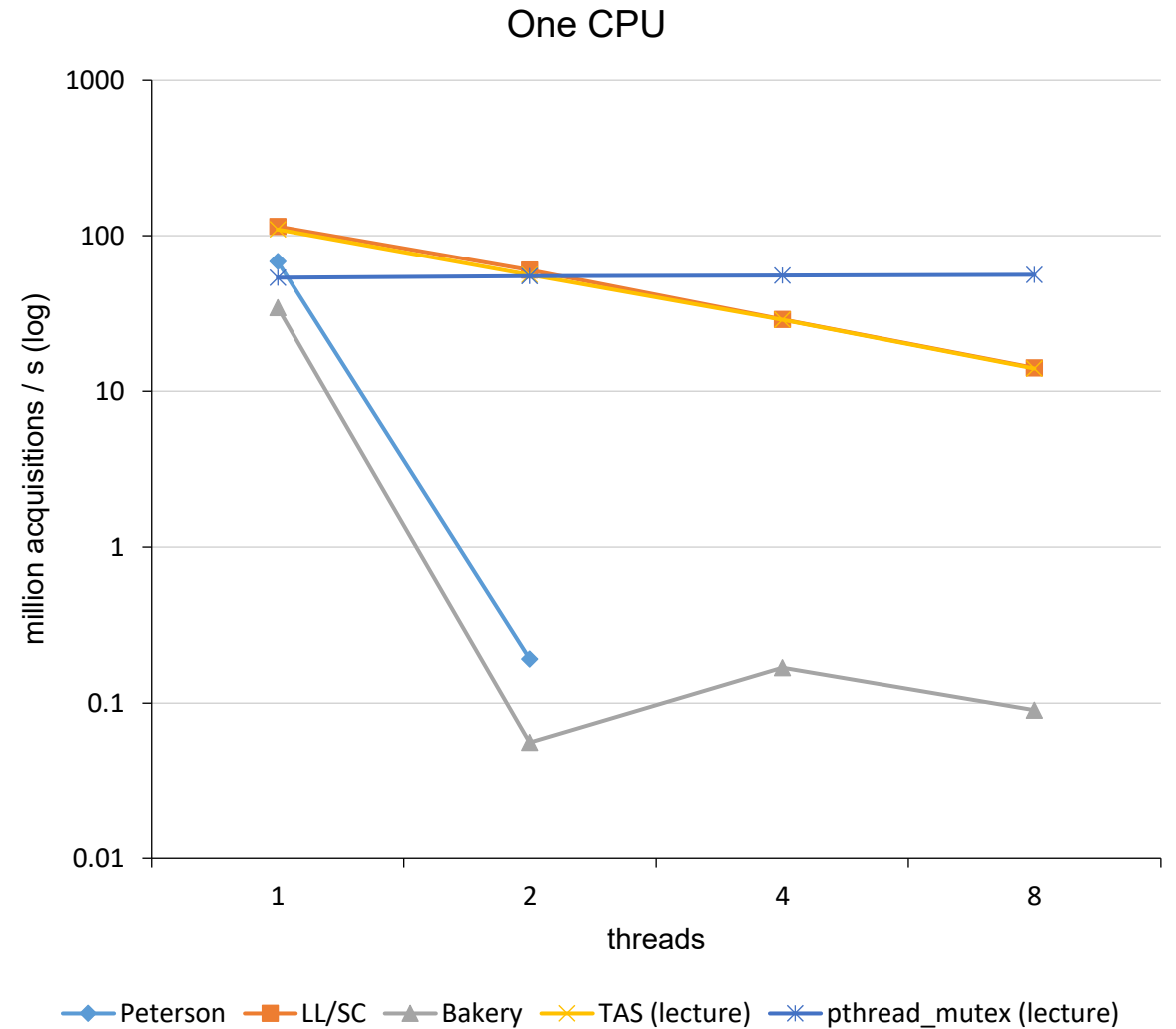
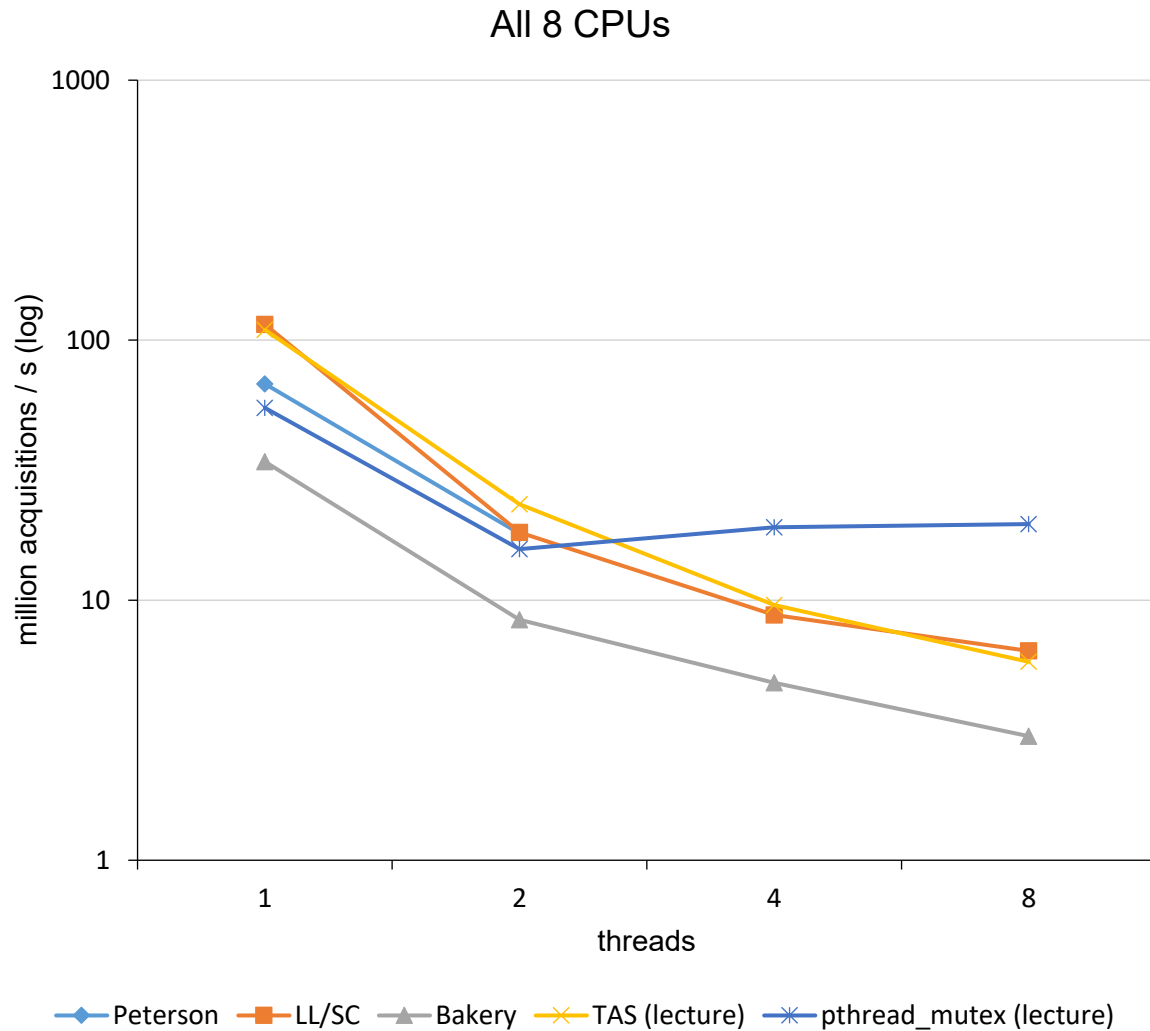
- First-come, first-served: fairness 1.00 / 0.93 / 0.97 at 2/4/8 threads (all CPUs) — the fairest lock at 8 threads
- Limitations:
 - $O(N)$ time and $O(N)$ memory per lock
 - Numbers grow without bound while threads keep contending (overflow)
 - Needs memory fences on modern CPUs; spins instead of sleeping
 - Collapses on one CPU: 90 k acq/s at 8 threads

Comparison

67070501058

Chanya Poolketkij

Throughput: All CPUs vs One CPU



Summary

Lock	Needs	Mutual excl.	Deadlock-free	Fairness	acq/s, all 8 CPUs	acq/s, one CPU
Peterson (2 thr)	loads/stores + fence	yes	yes	0.99	18 M	192 k
LL/SC (8 thr)	LL/SC instructions	yes	yes	0.45	6.4 M	14 M
Bakery (8 thr)	loads/stores + fence	yes	yes	0.97	3.0 M	90 k
TAS (8 thr)	test-and-set	yes	yes	0.83	5.8 M	14 M
pthread_mutex (8 thr)	atomics + futex	yes	yes	0.89	20 M	56 M

- Fairness = fewest ÷ most acquisitions per thread; medians of 3 runs of 1 s
- Peterson and Bakery give mutual exclusion on x86 only with the fence

Conclusion

- All three give mutual exclusion and no deadlock — Peterson and Bakery only with fences on modern CPUs
- Software locks (Peterson, Bakery): fair (bounded waiting), but slow and limited (2 threads / $O(N)$)
- Hardware LL/SC: one short loop, fast, but no fairness and not on x86
- Spinning on one CPU wastes time slices; sleeping locks (futex, pthread_mutex) avoid it

References

- R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, ch. 28 “Locks”
- CPE 333 Lecture 8: Locks Mechanism
- G. L. Peterson, “Myths About the Mutual Exclusion Problem”, Information Processing Letters 12(3), 1981
- L. Lamport, “A New Solution of Dijkstra's Concurrent Programming Problem”, CACM 17(8), 1974
- Arm Architecture Reference Manual for A-profile architecture: LDAXR, STXR
- Code and raw results: PS08/src, PS08/results